

VOID BREACH

Spécification technique de la vertical slice -
Version 0.3

Statut : architecture proposée et révisée

Document parent : Design Document v0.3

Moteur : S&box - Langage : C#

Support initial : 2 à 6 joueurs

Documentation S&box vérifiée le 25 juillet 2026

25 juillet 2026

Table des matières

- 1. Objectif de cette spécification
- 2. Paramètres de référence de la vertical slice
- 3. Fondations S&box retenues
- 4. Principes d'architecture
- 5. Topologie d'exécution
- 6. Machines d'état principales
- 7. Structure proposée du projet
- 8. Objets et composants principaux
- 9. Système de joueurs
- 10. Système d'interaction
- 11. Santé, état à terre et ressources énergétiques
- 12. Inventaire, armes et munitions
- 13. Présentation et animations FPS/TPS
- 14. Consoles et progression des indices
- 15. PDA et système de signal
- 16. Factions et IA
- 17. Boss et objectifs
- 18. Alerte et directeur d'assaut
- 19. Extractions
- 20. Déconnexion et fin de partie
- 21. Matrice d'autorité réseau
- 22. Scènes et prefabs
- 23. Outils de debug indispensables
- 24. Tests techniques requis
- 25. Risques techniques
- 26. Ordre de développement recommandé
- 27. Premier jalon concret
- 28. Décisions techniques encore ouvertes
- 29. Références techniques officielles

1. Objectif de cette spécification

Ce document traduit les règles de gameplay de Void Breach en une architecture technique exploitable.

Il définit :

- les responsabilités des différents systèmes ;
- l'autorité réseau ;
- les états du raid ;
- les données configurables ;
- les composants proposés ;
- les objets réseau nécessaires ;
- les interactions entre les systèmes ;
- le fonctionnement des présentations FPS et TPS ;
- les dépendances de développement ;
- les risques techniques majeurs.

Les noms de classes proposés dans ce document sont des noms propres au projet. Ils ne doivent pas être confondus avec des classes déjà présentes dans l'API S&box.

L'API de S&box continuant d'évoluer, chaque utilisation concrète d'une classe moteur devra être vérifiée dans la documentation et dans l'éditeur au moment de son implémentation.

2. Paramètres de référence de la vertical slice

2.1 Format multijoueur

La vertical slice doit fonctionner avec :

- 2 joueurs au minimum technique, répartis dans deux équipes distinctes ;
- 4 joueurs pour le test de référence, répartis en 2 équipes de 2 ;
- 6 joueurs au maximum initial, répartis en 3 équipes de 2 ;
- 2 joueurs maximum par équipe dans cette version.

Le nombre de joueurs et la taille d'équipe doivent provenir d'une configuration, et non être codés en dur dans les systèmes de raid.

2.2 Règles de spawn

Chaque équipe doit recevoir un point de spawn distinct.

Le système doit interdire :

- deux équipes au même point exact ;
- les collisions immédiates ;
- un avantage systématique vers la zone réelle du boss ;
- une réutilisation trop fréquente des mêmes départs.

Il ne doit pas imposer une distance minimale importante. Deux équipes peuvent commencer dans des zones adjacentes ou dans une même macro-zone.

2.3 Présentation fonctionnelle FPS et TPS

La vertical slice exclut les animations artistiques définitives, mais exige un pipeline fonctionnel :

- présentation FPS pour le propriétaire ;
- présentation TPS pour les autres joueurs ;
- ViewModel et WorldModel pour les objets tenus ;
- locomotion, tirs, rechargements, interactions, mise à terre et mort lisibles avec des placeholders.

Ce pipeline doit être validé dans le premier jalon technique, avant le développement de la boucle complète du raid.

2.4 Confidentialité des informations de console

Avant son propre scan, une équipe peut savoir qu'au moins une autre équipe a utilisé la console.

L'information transmise est uniquement un booléen :

- aucun accès extérieur détecté ;
- accès extérieur détecté.

Aucune donnée temporelle, identité, quantité, direction ou position ne doit être calculée ni envoyée au client. Cette restriction est permanente.

3. Fondations S&box retenues

La présente version s'appuie sur la documentation officielle S&box vérifiée le 25 juillet 2026.

S&box fournit actuellement :

- un système de scènes composé de GameObject, Component, prefabs et GameObjectSystem ;
- un PlayerController physique avec entrée, caméra FPS/TPS, animation compatible Citizen et événements d'utilisation ;
- un système officiel d'inventaire et d'armes fondé notamment sur BaseInventoryComponent et BaseCombatWeapon ;
- une séparation entre ViewModel FPS et WorldModel TPS ;
- des armes host-authoritative et owner-predicted ;
- un chargeur courant Clip1 et une réserve de munitions portée par l'inventaire ;
- des objets réseau, propriétés synchronisées, RPC et filtres de destinataires ;
- un système de navigation Recast ;
- des interfaces ScreenPanel, WorldPanel et Razor ;
- le lancement d'instances clientes supplémentaires pour les tests ;
- le support des serveurs dédiés ;
- des ressources Citizen et des modèles/animations de première personne réutilisables comme placeholders.

Le sample FPS/armes officiel et les ressources d'armes Facepunch doivent être évalués dans le premier jalon. Ils servent de base au pistolet, au ViewModel, au WorldModel, à l'inventaire et au comportement de tir avant toute abstraction propre à Void Breach.

Ces systèmes constituent des points de départ, pas une garantie que toutes les règles du projet soient déjà couvertes. Chaque API concrète doit être vérifiée dans la version du moteur utilisée au moment du code.

La vertical slice doit privilégier les composants officiels lorsqu'ils répondent au besoin, notamment pour :

- le contrôleur de joueur ;
- l'inventaire et le changement d'objet ;
- les armes, munitions et rechargements ;
- le chargeur courant et la réserve de munitions ;
- les ViewModels et WorldModels ;
- la navigation ;
- le réseau de base.

Les règles spécifiques au raid, aux équipes, aux consoles, au PDA, à la santé, au quota de taille des armes, aux objectifs et aux extractions restent propres au projet.

4. Principes d'architecture

4.1 Autorité de l'hôte

Toutes les décisions de gameplay importantes sont validées par l'hôte ou le serveur :

- configuration et phase du raid ;
- équipes et spawns ;
- zone et état réel du boss ;
- scans de consoles et progression privée des équipes ;
- dégâts définitifs et santé ;
- mise à terre, réanimation et mort ;
- possession et extraction des objectifs ;
- plan caché et exécution des incidents ;
- résultats du raid.

Le client peut prédire ou présenter une action immédiatement, mais ne peut pas en confirmer seul le résultat.

Cette autorité ne signifie pas que le mouvement est automatiquement anti-triche. Le modèle de propriété et de réplication du PlayerController devra être évalué séparément.

4.2 Présentation locale réactive

Les éléments purement visuels peuvent commencer immédiatement côté client :

- mouvement de caméra ;
- animation FPS de tir ;
- ouverture du PDA ;
- affichage initial d'une barre d'interaction ;
- effets d'interface ;
- sons personnels.

Le serveur conserve toutefois l'autorité sur le résultat.

Exemple :

1. le joueur appuie pour scanner une console ;
2. la progression apparaît immédiatement ;
3. l'hôte valide la distance et les conditions ;
4. l'hôte démarre l'interaction officielle ;
5. l'hôte confirme ou annule le résultat.

4.3 Données plutôt que code spécifique

Les différences entre armes, boss, extractions et interactions doivent être définies autant que possible par des ressources configurables.

Exemples :

- un boss ne doit pas nécessiter un nouveau gestionnaire de raid ;
- un camion et un train doivent partager le même socle d'extraction ;
- un medkit et une réanimation doivent partager le même moteur d'interaction ;
- une arme ne doit pas imposer une nouvelle logique d'inventaire.

4.4 Pas de gestionnaire omnipotent

Le projet ne doit pas contenir un unique GameManager contrôlant :

- les joueurs ;
- les armes ;
- les IA ;
- le boss ;
- les extractions ;
- le PDA ;
- les interactions.

Un gestionnaire global peut coordonner l'état du raid, mais les comportements spécialisés restent dans leurs composants respectifs.

5. Topologie d'exécution

5.1 Hôte ou serveur

L'hôte exécute :

- la machine d'état du raid ;
- la composition des équipes ;

- le choix des spawns ;
- la sélection du boss ;
- la progression des équipes ;
- l'IA ;
- les dégâts définitifs ;
- les interactions officielles ;
- les objectifs ;
- les extractions ;
- les résultats.

Pour les premiers développements, une partie hébergée localement suffit.

Le support d'un serveur dédié doit rester possible sans modifier les règles fondamentales.

5.2 Client propriétaire

Le client propriétaire gère ou présente immédiatement :

- les entrées ;
- la caméra ;
- le déplacement selon le modèle de propriété du contrôleur retenu ;
- le ViewModel FPS ;
- l'interface ;
- le PDA ;
- les demandes d'interaction ;
- les effets prédits des armes ;
- les sons personnels.

Les résultats de gameplay restent validés par l'hôte.

5.3 Clients distants

Les clients distants reçoivent uniquement les données publiques nécessaires :

- déplacement et rotation des personnages ;
- modèle TPS et objet tenu ;
- états d'animation ;
- tirs et effets ;
- état à terre et mort ;
- interactions visibles ;
- représentation physique des objectifs ;

- événements publics du raid.

Ils ne doivent jamais recevoir les données privées d'une autre équipe :

- consoles déjà scannées ;
- zones éliminées ;
- zone du boss lorsqu'elle n'est connue que de cette équipe ;
- informations internes du PDA.

La Network Visibility ne doit pas être utilisée comme mécanisme de confidentialité : la documentation indique que les objets restent connus du client et que les RPC continuent d'être délivrés lorsqu'un objet est culled. Les informations privées doivent rester côté hôte et être envoyées uniquement aux connexions autorisées par RPC filtré ou mécanisme équivalent.

6. Machines d'état principales

6.1 État global du raid

```
WaitingForPlayers
PreparingRaid
Searching
Alert
Finishing
Completed
Resetting
```

WaitingForPlayers

- le serveur accepte les joueurs ;
- les équipes ne sont pas figées ;
- le raid n'est pas lancé.

PreparingRaid

- création des équipes ;
- sélection des points de spawn ;
- sélection de la zone et du point du boss ;
- sélection des extractions actives ;
- génération des ennemis initiaux ;
- initialisation des états privés d'équipe ;
- préparation du plan d'incidents caché, si cette stratégie est retenue.

Searching

- le boss est vivant ;

- les consoles sont utilisables selon la progression de chaque équipe ;
- les équipes recherchent la cible ;
- les extractions actives connues sont visibles sur le PDA.

L'utilisation des extractions avant la mort du boss reste un paramètre de design ouvert et ne doit pas être supposée par la machine d'état.

Alert

- le boss est mort ;
- les objectifs sont immédiatement récupérables, transportés ou déjà extraits ;
- les consoles ne fournissent plus d'indices ;
- les incidents d'extraction peuvent survenir ;
- les cycles d'assaut sont actifs ;
- les objectifs prélevés sont traçables.

Finishing

- aucun joueur n'est encore actif dans la zone ;
- les résultats sont finalisés ;
- les actions de raid sont bloquées.

Completed

- les résultats peuvent être présentés ;
- le raid attend son reset.

Resetting

- destruction des entités dynamiques ;
- remise à zéro des extractions et consoles ;
- nettoyage des équipes ;
- préparation d'un nouveau raid.

6.2 État de vie d'un joueur

Alive
Downed
Dead
Extracted
Disconnected

Un joueur Extracted, Dead ou Disconnected n'est plus actif dans la zone. Un joueur Downed reste actif tant qu'il peut encore être réanimé.

6.3 État d'un boss

La connaissance du boss ne doit pas être représentée par un état global susceptible de révéler qu'une autre équipe l'a localisé.

État serveur du boss :

Dormant
Engaged
Dead

État public minimal :

Alive
Dead

Connaissance privée par équipe :

Unknown
Located

L'état Engaged reste serveur ou contextuel. Il ne déclenche aucune annonce globale. La mort du boss rend l'information publique et déclenche la phase Alert.

6.4 État d'une extraction

État public :

Inactive
Available
Destroyed

État interne au serveur :

Inactive
Available
ScheduledForIncident
IncidentExecuting
Destroyed

Une extraction ScheduledForIncident reste publiquement Available jusqu'au début effectif de son incident.

6.5 État d'un objectif

Unavailable
Harvestable
BeingHarvested
Dropped

Carried
Extracted

Les deux objectifs possèdent des états indépendants. Les transitions sont validées par l'hôte.

7. Structure proposée du projet

```
Code/
├─ Core/
│   ├─ Raid/
│   ├─ Teams/
│   ├─ Zones/
│   └─ Factions/
├─ Player/
│   ├─ Controller/
│   ├─ Health/
│   ├─ Interaction/
│   ├─ Inventory/
│   ├─ Presentation/
│   └─ PDA/
├─ Combat/
│   ├─ Weapons/
│   ├─ Damage/
│   └─ StatusEffects/
├─ AI/
│   ├─ Perception/
│   ├─ Navigation/
│   └─ Combat/
```

```
├─ Spawning/
├─ Objectives/
│   ├─ Clues/
│   ├─ Boss/
│   └─ Bounties/
├─ Extraction/
├─ UI/
├─ Data/
└─ Debug/
```

Cette organisation est indicative. Elle doit suivre les besoins réels du projet et ne pas produire artificiellement des dizaines de dossiers vides.

8. Objets et composants principaux

8.1 Coordination du raid

RaidCoordinator

Responsabilités :

- posséder la phase globale du raid ;
- démarrer la préparation ;
- déclencher l'alerte ;
- vérifier la condition de fin ;

- lancer le nettoyage ;
- exposer les événements globaux.

Il ne contrôle pas directement les comportements des IA, armes ou extractions.

RaidConfiguration

Ressource ou objet de configuration contenant notamment :

- nombre maximal de joueurs ;
- taille maximale des équipes ;
- nombre d'extractions actives ;
- règle de destruction ;
- nombre d'objectifs ;
- paramètres d'assaut ;
- références vers les configurations de boss.

RaidRuntimeState

État public minimal répliqué :

- phase actuelle ;
- zone de mort du boss, uniquement après sa mort ;
- nombre de joueurs encore actifs ;
- état public des extractions.

Le seed complet du raid, la zone réelle du boss, les extractions planifiées pour incident et toute autre sélection cachée restent exclusivement côté hôte. Un seed permettant de reconstruire ces informations ne doit pas être synchronisé aux clients.

8.2 Équipes

TeamService

Responsabilités :

- créer les équipes ;
- affecter les connexions ;
- fournir un identifiant d'équipe ;
- déterminer les relations entre joueurs ;
- suivre les résultats.

TeamProgressState - serveur uniquement

Données :

- membres et connexions ;
- consoles déjà scannées ;
- zones encore possibles ;
- boss localisé ou non ;
- zone du boss lorsqu'elle est connue ;
- objectifs extraits ;
- récompense potentielle.

TeamPdaClientState - copie locale autorisée

Chaque membre reçoit seulement la vue nécessaire à son PDA. L'hôte envoie les mises à jour aux connexions de l'équipe par RPC filtré fiable ou mécanisme équivalent. Le client stocke une copie locale destinée à l'affichage.

Il ne faut pas créer un objet réseau global contenant les données privées en supposant que la Network Visibility les rend secrètes.

8.3 Zones

RaidZone

Composant placé sur chaque macro-zone.

Propriétés proposées :

- identifiant stable ;
- nom d'affichage ;
- volume de zone ;
- points de console ;
- points de boss ;
- points d'ennemis ;
- routes de patrouille ;
- entrées de renforts ;
- points d'extraction possibles ;
- zones adjacentes.

ZoneService

Responsabilités :

- recenser les zones ;
- vérifier leur configuration ;

- choisir la zone du boss ;
- fournir les zones possibles à chaque équipe ;
- détecter dans quelle zone se trouve un événement.

Validation éditeur

Une zone invalide doit produire un avertissement si elle ne possède pas :

- de console ;
- de point de boss ;
- de point de rencontre ou spawn d'ennemi.

9. Système de joueurs

9.1 Prefab du joueur

Hiérarchie indicative :

```

PlayerRoot (GameObject réseau)
├─ PlayerController
├─ TeamMemberComponent
├─ HealthComponent existant / adapté
├─ DownedComponent
├─ InteractionController
├─ Inventory / système officiel d'objets
├─ WeaponCarryComponent
├─ PlayerPresentation
├─ ThirdPersonPresentation
├─ FirstPersonPresentation
├─ PdaController
├─ ObjectiveCarrierComponent
├─ OptionalShieldComponent
└─ Audio

```

Les noms sont propres au projet, sauf les composants moteur explicitement vérifiés.

Le prefab doit fonctionner avec un modèle Citizen ou compatible pendant la vertical slice.

WeaponCarryComponent ne remplace pas l'inventaire officiel : il ajoute la règle métier des deux emplacements d'arme et du quota de taille cumulé.

OptionalShieldComponent peut rester absent tant que le système batterie/bouclier n'entre pas dans le périmètre jouable.

9.2 Contrôleur

Le PlayerController officiel et la configuration du sample FPS doivent être prototypés avant toute écriture d'un contrôleur personnalisé.

Le PlayerController fournit actuellement :

- un contrôleur physique ;
- des entrées optionnelles ;
- une caméra FPS/TPS optionnelle ;
- un animateur compatible Citizen ;
- des événements de saut, réception, caméra et utilisation.

La première étape consiste à le configurer pour obtenir :

- mouvement réactif ;
- sprint ;
- saut ;
- accroupissement si retenu ;
- vitesse adaptée ;
- blocage des actions selon l'état du joueur ;
- changement vers le PDA et les autres objets ;
- désactivation du contrôle normal pendant l'état à terre.

Les événements d'utilisation du contrôleur peuvent servir d'adaptateur d'entrée au système d'interaction. La logique de maintien, d'annulation et de validation serveur reste propre à Void Breach.

Un contrôleur personnalisé ne doit être envisagé que si un besoin essentiel est réellement bloqué.

10. Système d'interaction

10.1 Interface commune

Une interface propre au projet, par exemple IWorldInteractable, expose :

- le texte de l'action ;
- la durée ;
- les conditions ;
- la politique de mouvement ;
- les causes d'annulation ;
- la disponibilité pour un joueur donné ;
- le résultat attendu.

La durée et les règles peuvent varier entre plusieurs actions visant une même cible. Un joueur à terre peut par exemple proposer une réanimation rapide avec ressource et un CPR sans ressource.

10.2 Détection

Le client propriétaire détermine la cible visée depuis la caméra, soit par une trace propre au projet, soit en s'intégrant au système d'utilisation du PlayerController.

La détection doit prendre en compte :

- portée configurable ;
- ligne de visée ;
- filtrage des cibles ;
- priorité en cas de plusieurs interactables ;
- disponibilité apparente pour le joueur ;
- liste des actions actuellement possibles sur la cible.

Le client n'exécute pas le résultat. Il demande à l'hôte de démarrer une action identifiée sur une cible identifiée. L'hôte retrouve ou valide la cible et ne doit pas faire confiance à une référence arbitraire sans contrôle de distance et de contexte.

10.3 Validation serveur

L'hôte vérifie :

- que le joueur est dans un état autorisé ;
- que la cible existe ;
- que la distance est valide ;
- que la ligne de vue est acceptable ;
- que la cible autorise cette action pour ce joueur ;
- que l'équipement ou la ressource requis est disponible ;
- que la cible n'est pas déjà réservée ;
- que la phase du raid autorise l'action.

10.4 Session d'interaction

Chaque joueur possède au maximum une interaction longue active.

Données :

- cible ;
- action sélectionnée ;

- heure de début côté serveur ;
- durée ;
- politique de déplacement ;
- progression ;
- conditions d'annulation ;
- réservation de cible ;
- ressource à consommer en cas de réussite.

La progression de l'interface peut être calculée à partir de l'heure de début et de la durée, plutôt que synchronisée chaque frame.

La ressource est consommée à la réussite par défaut. Une consommation sur interruption ne doit être ajoutée que si une définition le demande explicitement.

10.5 Politiques configurables

```
MovementPolicy
- Free
- WalkOnly
- Immobile
```

Paramètres complémentaires :

- sprint autorisé ;
- rotation de caméra autorisée ;
- saut autorisé ;
- annulation sur dégâts ;
- annulation sur éloignement ;
- annulation si le joueur regarde ailleurs ;
- interaction exclusive ;
- nombre maximal d'utilisateurs ;
- conservation ou non de la progression partielle.

10.6 Cas particuliers

Porte

- durée : 0 ;
- déplacement libre ;
- résultat immédiat.

Console

- interaction maintenue ;
- joueur immobile ;

- exclusive à un joueur à la fois ;
- annulation sur dégâts ou éloignement.

Réanimation rapide

- durée initiale : 3 secondes ;
- requiert Health Vial ou Personal Medkit ;
- joueur immobile ;
- cible réservée ;
- drain de HardMaxHealth suspendu pendant la session valide ;
- annulation sur dégâts, déplacement, rupture de distance ou mort de la cible.

CPR

- durée initiale : 10 secondes ;
- aucune ressource ;
- mêmes règles de réservation et d'annulation que la réanimation rapide ;
- drain de HardMaxHealth suspendu pendant la session valide.

Combine Medkit

- durée initiale : 3 à 5 secondes à tester ;
- restaure HardMaxHealth si la cible est Alive ou Downed avec HardMaxHealth supérieur à zéro ;
- refusé si la cible est Dead ;
- ne termine pas automatiquement la réanimation sauf définition future explicite.

Personal Medkit

- équipement actif ;
- interaction maintenue ;
- marche autorisée ;
- sprint et tir interdits ;
- annulation sur dégâts selon l'équilibrage.

Objectif

- durée de deux à trois secondes ;
- objet réservé pendant le prélèvement ;
- deux objectifs distincts récupérables simultanément.

Batterie ou dispositif alimentable

- hors minimum initial ;
- une action réussie consomme une batterie entière ;
- aucune charge partielle n'est suivie.

11. Santé, état à terre et ressources énergétiques

Le projet possède déjà des classes et du code de santé. La première tâche n'est pas de réécrire ce système, mais de l'auditer, de le tester en multijoueur et de l'aligner sur les règles ci-dessous.

Le document spécialisé **01-health-and-energy.md** fait autorité pour les règles détaillées.

11.1 Invariant de santé

```
CurrentHealth <= SoftMaxHealth <= HardMaxHealth
```

Données principales :

- CurrentHealth ;
- SoftMaxHealth ;
- HardMaxHealth ;
- StartingHardMaxHealth ;
- état de vie ;
- effets temporaires actifs ;
- pile de modificateurs du drain de HardMaxHealth ;
- dernière source de dégâts.

Toute modification de SoftMaxHealth ou HardMaxHealth doit immédiatement maintenir l'invariant. L'hôte applique les changements définitifs.

11.2 Effets temporaires sur SoftMaxHealth

Tant que le joueur est vivant, un effet de feu, poison ou autre définition compatible peut :

- réduire SoftMaxHealth selon un débit configurable ;
- limiter immédiatement CurrentHealth ;
- enregistrer une vitesse de récupération ;
- récupérer progressivement vers HardMaxHealth lorsque l'effet cesse.

Lorsque le joueur est à terre, le drain spécifique de SoftMaxHealth s'arrête. Le statut actif ajoute à la place un modificateur au drain de HardMaxHealth.

11.3 Passage à terre

Lorsque CurrentHealth atteint zéro :

1. si HardMaxHealth est déjà nul, passer à Dead ;
2. sinon passer à Downed ;
3. verrouiller CurrentHealth à zéro ;
4. désactiver les armes et interactions ordinaires ;
5. démarrer le drain autoritaire de HardMaxHealth ;
6. activer les interactions de réanimation ;
7. continuer à appliquer les modificateurs de drain.

Un joueur Downed reste actif pour la condition de fin de raid tant que HardMaxHealth est supérieur à zéro.

11.4 Pile de modificateurs du drain

$$\text{EffectiveDrainRate} = \text{BaseDrainRate} \times \text{Product}(\text{ActiveModifiers})$$

Un modificateur contient au minimum :

- identifiant de source ;
- multiplicateur ;
- durée ou condition de validité ;
- politique de cumul ;
- origine optionnelle pour le debug.

Le système doit accepter des origines internes, externes ou environnementales sans coder leur nature dans HealthComponent.

11.5 Réanimation

Toute réanimation utilise le système commun d'interaction et peut être effectuée par n'importe quel joueur vivant, quelle que soit son équipe.

Méthodes initiales :

Méthode	Ressource	Durée initiale
FastRevive	Health Vial ou Personal Medkit	3 s
CPR	Aucune	10 s
RestoreHardMax	Combine Medkit	3 à 5 s à tester

Pendant une session de réanimation valide et maintenue :

- le drain de HardMaxHealth est suspendu ;
- la cible est réservée si l'interaction est exclusive ;
- les conditions d'annulation sont évaluées par l'hôte.

À l'interruption, le drain reprend immédiatement.

À la réussite d'une réanimation :

```
WakeHealth = max(HardMaxHealth x WakePercent, MinWakeHP)
CurrentHealth = min(WakeHealth, SoftMaxHealth)
State = Alive
```

La réanimation ne change jamais l'équipe ou les relations de faction.

11.6 Mort définitive et Combine Medkit

Lorsque HardMaxHealth atteint zéro :

- le joueur passe à Dead ;
- toute interaction de réanimation est annulée ;
- le Combine Medkit ne peut plus le ressusciter ;
- les objectifs transportés sont déposés selon leurs règles.

Le Combine Medkit peut restaurer HardMaxHealth d'un joueur vivant ou Downed tant que cette valeur reste supérieure à zéro. Il ne remplace pas automatiquement la réanimation : il peut seulement restaurer le plafond et acheter du temps.

11.7 Soins progressifs

Les Health Vials, le Personal Medkit et les Health Stations restaurent CurrentHealth par interaction maintenue et restent plafonnés par SoftMaxHealth.

Le Personal Medkit possède un réservoir configurable et utilise un emplacement médical dédié.

11.8 Batterie et bouclier optionnel

Il n'existe pas de gilet, de tier d'armure ou de pipeline d'absorption par veste.

Une batterie est un objet discret sans charge partielle. Une utilisation consomme l'objet complet.

L'architecture peut ultérieurement permettre :

- d'ajouter un bouclier temporaire au joueur ;
- d'alimenter un dispositif du monde ;
- d'ouvrir une salle bonus ;
- de participer à l'activation d'une extraction secrète.

Ce système ne fait pas partie du minimum obligatoire. Aucun composant de bouclier ou HUD associé ne doit bloquer la boucle principale.

12. Inventaire, armes et munitions

Le document spécialisé **02-weapons-v0.3.md** fait autorité pour les règles détaillées.

12.1 Utilisation du système S&box

Le système officiel d'inventaire et d'armes doit servir de base à la vertical slice.

La documentation officielle décrit notamment :

- BaseInventoryComponent pour les objets et slots ;
- BaseCombatWeapon pour les armes, outils et objets tenus ;
- cadence, ballistique, dispersion, munitions et rechargement configurables ;
- ViewModel et WorldModel distincts ;
- déploiement, rangement et changement d'objet ;
- chargeur courant Clip1 ;
- réserve de munitions portée par l'inventaire ;
- prédiction du tir par le propriétaire et validation par l'hôte.

Avant d'ajouter des abstractions propres au projet, un prototype doit vérifier :

- l'intégration du sample FPS au PlayerController ;
- le fonctionnement multijoueur ;
- les animations FPS/TPS ;
- la possibilité de représenter le PDA et le medkit comme outils ;
- la manière d'intercepter ou d'adapter les dégâts au système de santé existant ;
- le fonctionnement du chargeur et de la réserve standard.

12.2 Structure des emplacements

Structure initiale :

```
WeaponSlotA
WeaponSlotB
ThrowableSlot
MedicalSlot
UtilitySlot
PdaSlot (locked)
ObjectiveCarrier (special, outside normal item slots)
```

Les deux emplacements d'arme sont génériques. Une couche propre à Void Breach valide la taille cumulée des armes.

Le PDA :

- est permanent ;
- ne peut pas être jeté ;
- ne consomme pas de taille d'arme ;
- utilise un emplacement verrouillé.

Les objectifs utilisent leur logique de transport dédiée.

12.3 Quota de taille des armes

WeaponCarryComponent ou une règle équivalente vérifie :

- deux armes maximum ;
- une somme de CarrySize inférieure ou égale à MaximumWeaponCarrySize ;
- une valeur initiale de quota égale à 5.

Valeurs initiales :

Catégorie	CarrySize
Pistolet / revolver	1
Mêlée légère	1
Mêlée lourde	2
SMG	3
Fusil d'assaut	4
Fusil à pompe	4
Arbalète	4
RPG	5

La validation s'applique lors de :

- l'ajout à l'inventaire ;
- la prise d'une arme au sol ;

- l'échange d'emplacement ;
- le chargement d'un loadout ;
- toute future restauration d'équipement.

Il n'existe pas de stockage d'armes supplémentaire dans le raid. Si l'arme ne tient pas, le joueur doit échanger une arme équipée ou abandonner la prise.

12.4 Munitions standard

La vertical slice n'utilise pas de chargeurs persistants comme objets.

Chaque arme utilise :

- un chargeur ou une chambre courante ;
- une réserve de munitions dans l'inventaire ;
- un type de munition configuré ;
- un maximum de réserve configurable.

Le rechargement transfère la quantité nécessaire de la réserve vers le chargeur. Les munitions déjà présentes dans l'arme ne sont pas perdues.

Règles particulières configurables :

- rechargement unitaire du fusil à pompe ;
- rechargement unitaire de l'arbalète ;
- MustBeEmptyToReload pour les armes énergétiques ;
- réserve limitée pour le RPG.

12.5 Définitions de données

Une définition d'arme propre au projet peut étendre ou compléter la ressource officielle avec :

- catégorie ;
- CarrySize ;
- type de munition ;
- taille du chargeur ;
- maximum de réserve ;
- règles de rechargement ;
- dégâts et pénétration éventuelle de bouclier ;
- paramètres ADS ;
- tirs principal et secondaire ;
- références ViewModel et WorldModel.

Aucune nouvelle classe spécifique n'est nécessaire pour une variante conventionnelle si les données et assets suffisent.

12.6 Ramassage et échange au sol

Lorsqu'un joueur vise une arme au sol :

1. calculer si elle peut être ajoutée directement ;
2. si le quota ou les deux emplacements sont dépassés, proposer l'échange avec une arme équipée ;
3. valider l'échange côté hôte ;
4. déposer l'ancienne arme dans le monde avec son état public nécessaire ;
5. équiper ou ranger la nouvelle arme selon la règle choisie.

Le comportement doit être testable sans interface d'inventaire complexe.

13. Présentation et animations FPS/TPS

13.1 Principe général

Chaque objet tenu possède deux présentations :

Première personne

- visible uniquement par le propriétaire ;
- positionnée par rapport à la caméra ;
- optimisée pour la vue FPS ;
- composée des bras et de l'objet ;
- capable d'utiliser des animations détaillées.

Troisième personne

- visible par les autres joueurs ;
- attachée au personnage ;
- composée du modèle monde ;
- pilotée par les animations du corps complet.

Le système officiel S&box sépare ViewModel et WorldModel pour les armes. Les ressources officielles peuvent servir de placeholders, mais le PDA, les soins et les interactions particulières devront être testés séparément.

13.2 Source de vérité

Les animations ne doivent pas décider du gameplay.

Le gameplay produit des états et événements :

- arme équipée ;
- tir ;
- rechargement ;
- sprint ;
- interaction ;
- soin ;
- PDA sorti ;
- réanimation ;
- mise à terre ;
- mort.

Les présentations FPS et TPS interprètent ensuite ces informations.

13.3 Synchronisation

Il ne faut pas synchroniser en permanence le temps exact de chaque animation.

Il faut synchroniser :

- l'objet actif ;
- les états persistants ;
- le début des actions longues ;
- les événements ponctuels importants.

Exemple de tir :

1. le propriétaire joue immédiatement le tir FPS ;
2. le système d'arme transmet la revendication à l'hôte ;
3. l'hôte valide les dégâts ;
4. les clients distants jouent le tir TPS et les effets réseau.

13.4 États persistants proposés

```
ActiveItem
IsSprinting
IsAiming
IsReloading
IsInteracting
InteractionType
```

IsDowned
IsDead
IsCarryingObjective

Les tirs, impacts et gestes courts utilisent des événements ponctuels plutôt qu'une synchronisation continue de leur animation.

13.5 Animations minimales de la vertical slice

Locomotion TPS

- idle ;
- marche ;
- course ;
- sprint ;
- saut ;
- chute ;
- réception ;
- mise à terre ;
- mort.

Armes FPS

- déploiement ;
- rangement ;
- idle ;
- tir ;
- tir à vide ;
- rechargement ;
- sprint ou arme abaissée.

Armes TPS

- tenue pistolet ;
- tenue arme longue ;
- tir ;
- rechargement ;
- changement d'arme.

Équipements et interactions

- PDA tenu en FPS ;
- PDA visible en TPS ;
- medkit tenu ;

- animation générique de soin ;
- animation générique d'interaction ;
- animation de réanimation ;
- animation de prélèvement.

13.6 Stratégie de placeholder

Pour éviter de bloquer le développement :

- utiliser le modèle Citizen ;
- utiliser les armes officielles compatibles ;
- réutiliser les animations existantes ;
- utiliser une pose générique pour le PDA ;
- utiliser une animation générique de manipulation pour les consoles ;
- utiliser une animation simple accroupie ou agenouillée pour la réanimation ;
- accepter de légers défauts de contact des mains dans un premier temps.

Le but est de valider la lisibilité des actions, pas leur qualité cinématographique.

13.7 Risque principal

Les armes classiques sont relativement bien couvertes par les ressources existantes.

Les éléments les plus compliqués seront :

- le PDA tenu à deux mains ;
- les soins en mouvement ;
- la réanimation ;
- les prélèvements spécifiques aux boss ;
- les créatures non humanoïdes ;
- les interactions correctes entre mains et objets.

Ces actions doivent être prototypées tôt, mais ne nécessitent pas immédiatement des animations originales parfaites.

14. Consoles et progression des indices

14.1 ClueConsole

Chaque console possède :

- un identifiant stable ;

- sa zone ;
- son état global ;
- la liste serveur des équipes l'ayant scannée ;
- son interactable ;
- sa source de signal.

La liste complète des équipes ne doit jamais être transmise à un client.

14.2 Affichage avant interaction

Lorsqu'un joueur cible la console, l'hôte ou la vue client autorisée détermine les informations suivantes :

```
CanScan
AlreadyScannedByOwnTeam
HasBeenScannedByAnotherTeam
LockedBecauseBossLocated
DisabledBecauseBossDead
```

Le client ne reçoit que les valeurs nécessaires à son propre affichage.

Exemple : Scanner la console Trace d'un accès extérieur détectée

Aucune liste d'équipes ou donnée temporelle n'est envoyée.

14.3 Résolution du scan

À la fin de l'interaction :

1. l'hôte vérifie à nouveau les conditions ;
2. l'équipe est ajoutée à la liste serveur des scans ;
3. la console est ajoutée à la progression privée de l'équipe ;
4. les zones possibles sont recalculées ;
5. une vue PDA mise à jour est envoyée uniquement aux membres autorisés ;
6. si une seule zone reste, le boss est localisé pour cette équipe.

La même équipe ne peut pas bénéficier deux fois de la même console.

14.4 Méthode de réduction

Pour éviter un résultat incohérent, le serveur connaît toujours la zone réelle du boss.

Chaque console fournit une élimination déterministe ou semi-aléatoire de zones incorrectes.

Elle ne doit jamais :

- éliminer la vraie zone ;
- réintroduire une zone déjà éliminée ;
- donner une progression différente aux membres d'une même équipe.

15. PDA et système de signal

15.1 PDA comme item actif

Le PDA utilise le même système de déploiement que les autres objets tenus.

Lorsqu'il est actif :

- les armes sont rangées ;
- le ViewModel du PDA est affiché ;
- le WorldModel est visible en TPS ;
- le joueur peut sprinter ;
- il ne peut pas tirer.

15.2 Interface

Deux approches sont possibles pour la vertical slice.

WorldPanel réellement rendu sur le PDA

Avantage :

- diégèse forte.

Risques :

- lisibilité ;
- interaction ;
- coût de mise au point ;
- comportement en mouvement.

Interface écran alignée visuellement sur le PDA Le modèle est tenu devant la caméra, tandis qu'un ScreenPanel ou panneau Razor est placé visuellement au-dessus de l'écran de l'objet.

Avantages :

- lisibilité ;
- mise en œuvre plus simple ;

- itération UI plus rapide.

La seconde approche est recommandée pour le premier prototype. Le choix reste réversible.

15.3 Sources de signal

```
Clue
Boss
ObjectiveA
ObjectiveB
```

Une source définit au minimum :

- sa position ;
- son profil de puissance ;
- les phases où elle est active ;
- les équipes autorisées à la détecter ;
- son état de résolution ;
- son identifiant de signature.

Les consoles déjà résolues par une équipe ne sont plus des sources valides pour celle-ci.

15.4 Recherche avant le boss

Le PDA choisit une source non résolue pertinente, initialement la source produisant le signal reçu le plus fort.

Les consoles et le boss doivent partager :

- la même catégorie de signature ;
- la même présentation UI ;
- les mêmes règles générales de cône et de force.

Des multiplicateurs internes peuvent être configurés pour équilibrer la détection, mais ils ne doivent jamais permettre au joueur d'identifier le type de source.

Le joueur peut donc être dirigé :

- vers une console proche ;
- vers le boss s'il domine le signal ;
- vers une source située dans la même zone sans connaître sa nature.

15.5 Calcul du cône

Le système calcule :

- direction réelle ;
- distance ;
- force approximative ;
- imprécision angulaire.

Plus la distance augmente :

- plus le cône est large ;
- plus la force est vague ;
- plus la mise à jour peut être lente.

Aucune coordonnée exacte n'est exposée à l'interface.

15.6 Traque des objectifs

Un objectif commence à émettre après sa première récupération.

S'il est ensuite :

- lâché ;
- abandonné ;
- posé sur un cadavre ;
- perdu après déconnexion ; il continue d'émettre.

Le PDA suit donc l'objet, mais l'émission n'est activée qu'après son prélèvement initial.

16. Factions et IA

16.1 Relations

Le système de factions doit représenter :

- les équipes de joueurs ;
- le Cartel ;
- les créatures Xen ;
- les boss.

Source	Cible	Relation initiale
Joueur	Équipe adverse	Hostile
Joueur	Même équipe	À décider / configurable
Cartel	Joueur	Hostile

Cartel Xen	Xen Joueur	Hostile Hostile
Source Xen	Cible Cartel	Relation initiale Hostile

La coopération entre équipes de joueurs ne modifie jamais cette matrice : elle reste officieuse.

16.2 Autorité IA

L'IA est simulée uniquement par l'hôte.

Les clients reçoivent :

- position ;
- rotation ;
- état d'animation ;
- cible visible si nécessaire ;
- attaques et effets.

16.3 Navigation

Les ennemis humanoïdes et le zombie simple utilisent la navigation Recast de S&box. Le NavMesh et ses agents fournissent le déplacement de base, tandis que la physique reste utilisée pour les interactions précises avec le sol et les obstacles.

16.4 Architecture IA minimale

```

AICharacter
├─ FactionComponent
├─ HealthComponent
├─ PerceptionComponent
├─ TargetSelectionComponent
├─ NavigationComponent
├─ CombatBehaviour
└─ Presentation

```

Le zombie et le soldat partagent :

- perception ;
- sélection de cible ;
- factions ;
- santé ;
- navigation.

Ils diffèrent principalement par leur comportement de combat et leur présentation.

17. Boss et objectifs

17.1 BossController

Responsabilités :

- comportement du boss ;
- gestion des renforts pendant le combat ;
- notification de découverte ;
- notification de mort ;
- déclenchement des deux objectifs.

Le boss ne déclenche pas directement les extractions. Il notifie le RaidCoordinator , qui démarre la phase d'alerte.

17.2 Découverte directe

Une équipe peut localiser le boss si :

- elle termine suffisamment de scans ;
- un membre entre dans un volume de découverte ;
- le boss attaque un membre ;
- un membre obtient une détection visuelle validée.

La découverte directe ne révèle rien aux autres équipes.

17.3 Objectifs

Deux objets réseau distincts sont créés ou activés.

Chaque objectif :

- peut être réservé par une interaction ;
- peut être porté par un seul joueur ;
- peut être lâché ;
- peut être extrait ;
- survit à la déconnexion du porteur.

17.4 Résultat d'équipe

Lorsqu'un objectif est validé par une extraction :

- il passe à l'état Extracted ;
- il est attribué à l'équipe du porteur ;
- le compteur de cette équipe augmente.

Objectifs extraits par l'équipe	Résultat
0	0%
1	50 %
2	100 %

18. Alerte et directeur d'assaut

18.1 AlertDirector

Responsabilités :

- démarrer les cycles d'assaut ;
- choisir les points de renfort ;
- respecter un budget maximal ;
- alterner pression et accalmie ;
- appliquer le profil post-boss.

18.2 Budget

Le directeur ne doit pas créer une quantité infinie d'ennemis.

Il utilise :

- nombre maximal d'ennemis vivants ;
- coût par type d'ennemi ;
- délai entre vagues ;
- plafond de difficulté ;
- distance minimale aux joueurs ;
- points d'entrée autorisés.

19. Extractions

19.1 ExtractionPoint

Composant commun au train et au camion.

Propriétés :

- identifiant ;
- type visuel ;
- statut ;
- volume d'extraction ;
- présentation intacte ;
- présentation détruite ;
- profil d'incident ;
- emplacement affiché sur le PDA.

19.2 Sélection des extractions actives

Pendant PreparingRaid :

1. recenser tous les emplacements possibles ;
2. choisir le nombre requis ;
3. vérifier leur répartition ;
4. activer les extractions retenues ;
5. rendre les autres inactives ;
6. publier les extractions actives sur les PDA.

19.3 Plan d'incident

À l'entrée dans la phase Alert, l'hôte détermine :

- le nombre d'extractions à neutraliser, entre zéro et environ 70 % ;
- les extractions concernées ;
- l'ordre des incidents ;
- leurs délais ;
- leur profil visuel ou sonore.

Le plan peut être pré-calculé pendant PreparingRaid ou décidé à la mort du boss, mais il reste entièrement caché.

Contraintes :

- toujours au moins une extraction disponible ;
- aucune annonce préalable sur le PDA ;
- incidents espacés ;
- aucune dépendance à la récupération des deux objectifs ;

- objectifs immédiatement récupérables dès la mort du boss.

19.4 Exécution

Lorsqu'un incident survient :

1. jouer la télégraphie locale ;
2. désactiver la zone d'extraction ;
3. jouer l'effet ;
4. remplacer éventuellement le modèle intact ;
5. passer au statut Destroyed ;
6. mettre à jour tous les PDA.

19.5 Session d'extraction

Une extraction peut traiter un joueur seul ou plusieurs joueurs présents dans son volume.

Paramètres à tester :

- durée ;
- interruption en cas de sortie ;
- interruption en cas de dégâts ;
- extraction individuelle ou groupée ;
- autorisation ou non avant la mort du boss.

L'architecture doit permettre une extraction avec zéro, un ou deux objectifs. Le résultat d'équipe vaut respectivement 0 %, 50 % ou 100 %.

La disponibilité des extractions pendant la phase Searching reste une décision de design ouverte et doit être contrôlée par configuration.

20. Déconnexion et fin de partie

20.1 Déconnexion

Lorsqu'une connexion disparaît :

- le joueur passe à Disconnected ;
- ses objectifs sont déposés ;
- ses interactions sont annulées ;
- il est retiré du compte des joueurs actifs ;
- ses objets importants restent récupérables.

La reconnexion en cours de raid est hors périmètre initial.

20.2 Détection de fin

Après chaque événement retirant potentiellement un joueur actif - mort, extraction ou déconnexion - le RaidCoordinator recompte les états de vie.

Si aucun joueur n'est Alive ou Downed, le raid entre dans Finishing.

21. Matrice d'autorité réseau

Système Entrées nécessaires Caméra Déplacement validations Tir FPS applique les	Client propriétaire Produit les entrées Contrôle local Exécuté selon la propriété du PlayerController Joue immédiatement la prédiction	Hôte / serveur Reçoit les demandes Aucun contrôle direct Reçoit la réplication ; supplémentaires à définir Valide les claims et
--	---	--

dégâts

Santé État à terre Interaction termine Console envoie la vue	Affiche Affiche Détection, demande et affiche Demande le scan	Modifie et décide Décide Valide, chronomètre et Modifie l'état serveur et
---	--	--

privée autorisée

PDA privé vérité et filtre Boss états cachés IA Objectifs transitions Extractions exécute les Résultats	Affiche une copie locale Affiche les données publiques Affiche Demande prélèvement, dépôt ou extraction Affiche l'état public Affiche	Conserve la source de les destinataires Simule et conserve les Simule Attribue et valide les Sélectionne, planifie et incidents Calcule
--	---	--

22. Scènes et prefabs

22.1 Scène de bootstrap

Contient :

- configuration réseau ;
- coordinateur du raid ;

- services globaux ;
- interface de développement.

22.2 Carte greybox

La carte greybox contient :

- géométrie statique ;
- zones ;
- consoles ;
- points de boss ;
- points d'IA ;
- extractions possibles ;
- NavMesh ;
- points de spawn.

La géométrie et les objets statiques communs peuvent être chargés de façon identique sur chaque client. Les objets dynamiques, états variables et entités créées pendant le raid utilisent le réseau selon leur besoin.

Le choix final entre un workflow principalement Hammer, Scene ou hybride doit être validé par un prototype de carte avant la production des six zones.

22.3 Prefabs principaux

- joueur ;
- zombie ;
- soldat ;
- boss ;
- console ;
- objectif ;
- camion ;
- train ;
- arme issue du sample FPS ;
- armes de la vertical slice ;
- PDA ;
- Health Vial ;
- Personal Medkit ;
- Combine Medkit ;
- Health Station ;

- batterie et dispositif alimentable uniquement lorsque cette fonctionnalité entre dans le périmètre.

23. Outils de debug indispensables

Une interface de développement doit pouvoir afficher :

- phase du raid ;
- seed côté développement uniquement ;
- composition des équipes ;
- zone réelle du boss ;
- progression privée de chaque équipe ;
- consoles scannées ;
- extractions actives ;
- extractions planifiées pour destruction ;
- état des deux objectifs ;
- porteurs ;
- nombre d'ennemis ;
- joueurs actifs ;
- CurrentHealth, SoftMaxHealth et HardMaxHealth ;
- vitesse effective de drain et modificateurs ;
- état Downed ou Dead ;
- deux armes équipées, CarrySize total et quota ;
- chargeur courant et réserve de munitions ;
- bouclier éventuel uniquement si le système est actif.

Ces informations ne sont visibles qu'en mode développement.

Des actions de debug doivent permettre de :

- tuer le boss ;
- simuler un scan ;
- passer en alerte ;
- détruire une extraction ;
- appliquer des dégâts ;
- réduire SoftMaxHealth ;
- mettre un joueur à terre ;
- modifier HardMaxHealth ;
- lancer ou annuler une réanimation ;
- restaurer HardMaxHealth avec le Combine Medkit ;

- attribuer une arme et modifier sa taille ;
- donner des munitions ;
- attribuer un objectif ;
- terminer ou réinitialiser le raid.

24. Tests techniques requis

24.1 Réseau

Tester progressivement avec :

- deux instances ;
- quatre instances ;
- six instances ;
- une partie hébergée par un joueur ;
- un serveur dédié lorsque la boucle principale fonctionne.

Le serveur dédié doit être testé séparément, notamment pour vérifier que le code ne dépend pas d'une caméra, d'un joueur local ou d'une interface présente sur l'hôte.

24.2 Cas critiques

- deux équipes scannent simultanément la même console ;
- deux joueurs réaniment la même cible ;
- un joueur ennemi réanime la cible sans changer d'équipe ;
- une réanimation est interrompue et le drain reprend ;
- le Combine Medkit restaure HardMaxHealth juste avant zéro ;
- le Combine Medkit est refusé après le passage à Dead ;
- SoftMaxHealth diminue sous CurrentHealth et provoque le clamp ;
- plusieurs modificateurs de drain se cumulent ;
- deux joueurs prélèvent les deux objectifs simultanément ;
- un porteur se déconnecte ;
- un porteur meurt dans une extraction ;
- une extraction est détruite pendant son utilisation ;
- aucune extraction n'est détruite ;
- une seule extraction survit ;
- le boss est découvert sans indice ;
- un joueur équipe AR + pistolet ;
- un joueur équipe pistolet + pistolet ;
- le système refuse SMG + SMG avec les tailles initiales ;

- un joueur échange une arme équipée avec une arme au sol ;
- le chargeur et la réserve restent cohérents après plusieurs rechargements ;
- tous les joueurs meurent ;
- le dernier joueur se déconnecte ;
- le raid est réinitialisé plusieurs fois.

25. Risques techniques

25.1 Animations FPS/TPS - risque élevé

Le système de base existe, mais les interactions personnalisées demanderont :

- des poses ;
- des transitions ;
- des modèles tenus correctement ;
- une synchronisation entre gameplay, FPS et TPS.

Réduction du risque :

- commencer par le sample FPS et les ressources officielles ;
- employer Citizen et les armes officielles ;
- limiter les actions personnalisées ;
- accepter des placeholders.

25.2 Autorité réseau des interactions - risque élevé

Une interaction longue doit rester correcte malgré :

- latence ;
- annulation ;
- déplacement ;
- dégâts ;
- cible détruite ;
- déconnexion.

Réduction du risque :

- une seule session active par joueur ;
- validation serveur ;
- progression fondée sur un temps serveur ;
- tests multiciens précoces.

25.3 Intégration du code de santé existant - risque élevé

Le code existant peut avoir été conçu selon une API S&box, une autorité réseau ou des hypothèses différentes de la spécification actuelle.

Réduction du risque :

- audit avant modification ;
- tests unitaires ou de scène sur l'invariant des trois valeurs ;
- vérification de l'autorité réseau ;
- adaptation progressive plutôt qu'une réécriture automatique ;
- conservation des comportements validés par le document spécialisé.

25.4 Inventaire officiel et quota de taille - risque moyen

Le système officiel gère les objets et munitions, mais la règle des deux armes et du quota cumulé est propre à Void Breach.

Réduction du risque :

- spike avec le pistolet du sample ;
- ajout d'une validation métier mince autour de l'inventaire ;
- aucun stockage d'armes supplémentaire ;
- échange au sol testé avant l'arsenal complet.

25.5 IA multifaction - risque moyen à élevé

Les IA doivent :

- choisir leurs cibles ;
- changer de cible ;
- éviter des combats absurdes ;
- fonctionner avec plusieurs joueurs.

Réduction du risque :

- partager perception et ciblage ;
- commencer avec deux comportements simples ;
- limiter le nombre d'ennemis.

25.6 Production 3D - risque majeur à long terme

Le greybox et les assets temporaires suffisent pour la vertical slice.

Ils ne suffiront pas pour une version artistique complète comprenant :

- plusieurs créatures uniques ;
- plusieurs boss ;
- animations spécifiques ;
- armes originales ;
- véhicules détaillés.

Ce risque ne doit pas empêcher la validation du gameplay.

26. Ordre de développement recommandé

Phase 0 - Prototype technique FPS/TPS et sample officiel

Avant le véritable raid :

- créer le projet et la micro-scène ;
- ajouter ou reprendre le sample FPS/armes officiel ;
- intégrer un PlayerController et un personnage Citizen ;
- connecter un second client ;
- ajouter le pistolet du sample ;
- valider ViewModel FPS et WorldModel TPS ;
- valider tir, chargeur, réserve et rechargement ;
- valider le changement d'objet ;
- ajouter un PDA placeholder ;
- créer une interaction instantanée et une interaction maintenue.

Cette phase doit confirmer que la base de présentation, de réseau et d'inventaire est viable.

Phase 1 - Audit et intégration de la santé existante

- importer le code existant ;
- cartographier les classes et dépendances ;
- vérifier CurrentHealth, SoftMaxHealth et HardMaxHealth ;
- vérifier l'autorité réseau ;
- tester le passage Downed/Dead ;
- tester le drain et ses modificateurs ;
- intégrer les événements de dégâts au pistolet du sample.

Phase 2 - Infrastructure du raid

- connexion ;
- équipes ;
- spawns ;
- phases ;
- déconnexion ;
- fin et reset.

Phase 3 - Interaction commune

- ciblage ;
- instantané ;
- maintien ;
- annulation ;
- validation réseau ;
- politiques de mouvement.

Phase 4 - Inventaire et règles de transport

- deux emplacements d'arme ;
- CarrySize et quota ;
- slots d'équipement ;
- PDA verrouillé ;
- échange d'arme au sol ;
- réserve de munitions par type.

Phase 5 - Carte et zones

- six zones ;
- points configurables ;
- randomisation ;
- extractions possibles.

Phase 6 - PDA et consoles

- item PDA ;
- carte ;
- sources ;
- cône ;
- progression par équipe ;

- trace d'accès adverse.

Phase 7 - Combat, soins et arsenal

- pistolet stabilisé ;
- SMG ;
- fusil d'assaut ;
- fusil à pompe ;
- Health Vial ;
- Personal Medkit ;
- Combine Medkit ;
- réanimation rapide et CPR.

Phase 8 - IA

- factions ;
- zombie ;
- soldat ;
- combats croisés.

Phase 9 - Boss et objectifs

- boss ;
- renforts ;
- mort ;
- prélèvements ;
- transport ;
- signaux.

Phase 10 - Alerte et extractions

- directeur d'assaut ;
- incidents ;
- camion ;
- train ;
- extraction ;
- récompenses.

Phase 11 - Stabilisation

- tests à six joueurs ;
- équilibrage ;

- debug ;
- correction des blocages ;
- validation de la boucle complète.

La batterie, le bouclier et les dispositifs alimentables sont ajoutés seulement si la boucle principale est stable et si leur test apporte une décision utile.

27. Premier jalon concret

Le premier jalon n'est pas encore une carte de six zones.

Il s'agit d'une micro-scène contenant :

- un sol greybox ;
- deux points de spawn ;
- deux joueurs connectés ;
- un PlayerController configuré à partir des composants officiels ;
- un modèle Citizen visible en TPS ;
- une caméra FPS ;
- le pistolet du sample FPS/armes officiel ;
- un ViewModel et un WorldModel ;
- un chargeur courant et une réserve de munitions ;
- tir et rechargement visibles des deux côtés ;
- un PDA placeholder pouvant être équipé ;
- une caisse ou arme avec interaction instantanée ;
- une porte fonctionnelle ;
- une console avec interaction maintenue ;
- une interruption de l'interaction par déplacement.

Ce jalon permettra de valider simultanément :

- le réseau ;
- le contrôleur ;
- l'inventaire officiel ;
- le sample FPS ;
- les armes et munitions ;
- les animations FPS ;
- les animations TPS ;
- le changement d'équipement ;
- le futur système d'interaction.

Le code de santé existant peut être importé et audité dès que ce socle de tir et de dégâts permet un test fiable, mais le boss, les indices et les extractions ne doivent pas commencer avant stabilisation du jalon.

28. Décisions techniques encore ouvertes

Les points suivants ne doivent pas être figés implicitement par le code :

- utilisation des extractions avant la mort du boss ;
- extraction individuelle ou groupée ;
- dégâts entre membres d'une même équipe ;
- capacité d'un joueur à porter un ou deux objectifs ;
- modèle exact d'autorité et de validation du déplacement ;
- rendu du PDA en WorldPanel réel ou en interface écran alignée ;
- workflow de level design Hammer, Scene ou hybride ;
- quantité d'animations personnalisées nécessaires après les placeholders ;
- adaptation exacte du système officiel de dégâts au code de santé existant ;
- noms et responsabilités réels des classes de santé après audit ;
- valeurs finales du quota et des CarrySize ;
- règle vide-seulement des armes énergétiques ;
- implémentation ou non de la batterie et du bouclier dans la vertical slice étendue ;
- valeur du bouclier et présentation HUD si cette fonctionnalité est retenue.

Chaque décision ouverte doit être résolue par un petit prototype ou un playtest, puis reportée dans le document de conception.

29. Références techniques officielles

Documentation S&box vérifiée le 25 juillet 2026 :

- Scene, GameObjects et Components ;
- Player Controller ;
- Inventory et Weapons ;
- BaseCombatWeapon et chargeur Clip1 ;
- Weapon Models et ressources First Person Weapons ;
- Citizen Characters ;
- Networking ;
- RPC Messages et filtrage des destinataires ;

- Network Visibility ;
- Networked Objects ;
- Sync Properties ;
- Testing Multiplayer ;
- Dedicated Servers ;
- Navigation Recast ;
- UI et Razor Panels.

Ces références doivent être revérifiées avant l'implémentation d'un système important, car S&box évolue rapidement.